

QUESTION:

You just got some contract work with an Ecommerce company based in New York City that sells clothing online but they also have in-store style and clothing advice sessions. Customers come in to the store, have sessions/meetings with a personal stylist, then they can go home and order either on a mobile app or website for the clothes they want.

The company is trying to decide whether to focus their efforts on their mobile app experience or their website. They've hired you on contract to help them figure it out! Let's get started!

Explanation:

To answer this we should understand what is Linear Regression?

Linear regression is a foundational technique in machine learning used to predict a continuous numerical value. Technically, it models the relationship between a dependent variable (the output we want to predict) and one or more independent variables (the features or inputs) by fitting a linear equation—a straight line—to the observed data. The goal is to determine the optimal line that minimizes the error, or the difference, between the predicted and actual values.

In Simple Terms

In linear regression, we use an existing algorithm to create a model. We give the model some data so that it can learn the relationship between the input and the output.

To understand this more easily, imagine that we have **100 sets of data**. Each data set has an **input** and a corresponding **output**.

For example, we can divide these 100 data sets into two parts. We can use **80% of the data to train the model** and the remaining **20% to test the model**.

During the training process, we give the model **80 inputs and their corresponding 80 outputs**. By looking at these inputs and outputs, the model learns the relationship between them.

After training the model, we test whether the model has learned properly.

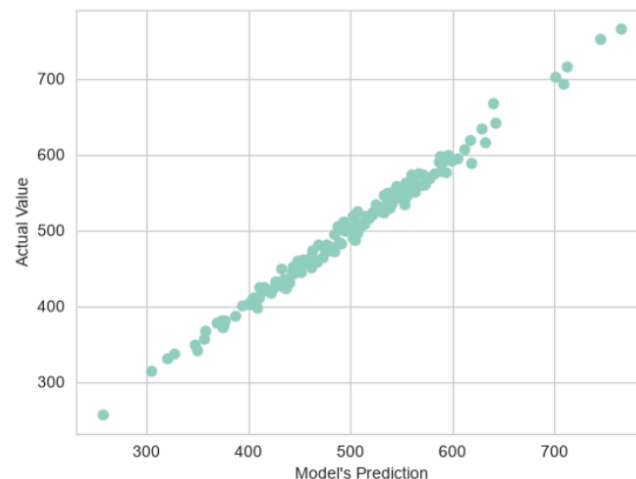
For testing, we give the trained model the remaining **20 inputs only**. We do not give the model the actual outputs for these 20 inputs. Instead, the model uses what it learned during training to predict the outputs.

As a result, the model produces **20 predicted outputs**.

However, we already know the actual outputs for those 20 inputs. Therefore, we can compare the **20 predicted outputs** with the **20 actual outputs**.

This allows us to see how well the model performs.

The graph below shows the relationship between the predicted and actual values. The x-axis represents the machine's predicted values, while the y-axis represents the actual values.



The closer the data points are to the line where the predicted value equals the actual value, the better the model performs. Since most points are close to this line, the model's predictions appear to be close to the actual values.

However, we can also use numerical metrics to evaluate the model's performance, which I will discuss later in this PDF.

So, the overall process has two main parts:

First, we **train the model** using 80 inputs and their corresponding 80 outputs.

Second, we **evaluate the model** by giving it 20 new inputs and comparing its predicted outputs with the actual outputs.

We can use different evaluation metrics to measure how well the model performs. These metrics will help us understand whether the model is reliable enough for our purpose. After evaluating the model and confirming that it performs well, we can use the model for prediction or analysis.

In this particular question, we use the model mainly for **analysis**. We are not using the model to predict future values. Instead, we use it to understand the relationship between the variables and analyze the data.

However, in the next question, you will learn how these models can also be used to make predictions.

So, in simple terms, the process is:

**Give the model data → Train the model → Test the model → Evaluate its performance
→ Use the model for analysis or prediction.**

Answer Code

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 customers = pd.read_csv("Ecommerce Customers")
6
7 pd.set_option('display.max_columns', None)
8 pd.set_option('display.width', 1000)
9
10 print(customers.head())
11 print('__'*500)
12 print(customers.describe())
13 print('__'*500)
14 print(customers.info())
15
```

Explanation of the Initial Code

Lines 1-4: Importing Necessary Libraries

These lines import the essential libraries used for data analysis and visualization.

Detailed Explanation of Remaining Code:

- **Line 5:** `customers = pd.read_csv("Ecommerce Customers")`
This command loads your dataset, which is stored in a file named "Ecommerce Customers," into a variable called **customers**. The **pd.read_csv** function converts that file into a format called a DataFrame, which acts like an organized table (similar to an Excel spreadsheet) that you can easily analyze using Python.
- **Line 7:** `pd.set_option('display.max_columns', None)`
This changes the settings for your pandas DataFrame. By default, if a table has many columns, pandas might hide some of them. Setting this option to **None** ensures that every column in your dataset is displayed when you print it.
- **Line 8:** `pd.set_option('display.width', 1000)`
This adjusts the layout of your display. It sets the width to 1,000 characters, which prevents the data from wrapping to new lines, making it much easier to read tables when they are printed to the console.
- **Lines 10, 12, 14:** `print(customers.head())`, `print(customers.describe())`, and `print(customers.info())`
These commands are used to inspect the data you just loaded:

- **customers.head():** Displays the first 5 rows of your table. This gives you a quick visual check to ensure the data loaded correctly and lets you see the names of your columns.
- **customers.describe():** Calculates statistical summaries for all numeric columns in your data (like the mean, minimum, maximum, and standard deviation). This is great for getting a "big picture" understanding of the numerical values in your dataset.
- **customers.info():** Provides a concise summary of the DataFrame structure, including the total number of rows, the data type of each column (e.g., text vs. numbers), and a count of how many non-null values are present. This helps you spot missing or incorrect data.

```

16 #=====
17 # TRY TO PREDICT AN ANSWER BY YOURSELVES USING BELOW GRAPHS
18 #=====
19 sns.set_palette("GnBu_d")
20 sns.set_style('whitegrid')
21
22 # More time on site, more money spent.
23 sns.jointplot(x='Time on Website',y='Yearly Amount Spent',data=customers)
24 plt.show()
25
26 #After reviewing graph we can predict that more time on app more spend
27 sns.jointplot(x='Time on App',y='Yearly Amount Spent',data=customers)
28 plt.show()
29
30 sns.jointplot(x='Time on App',y='Length of Membership',kind='hex',data=customers)
31 plt.show()
32
33 sns.pairplot(customers)
34 plt.show()
35
36 sns.lmplot(x='Length of Membership',y='Yearly Amount Spent',data=customers)
37 plt.show()
38

```

This new set of code is not part of the required module. It is only provided to give you an idea of how to illustrate and represent the graphs. You can use your own judgment to decide which modules are most suitable.

These graphs are related to my previous data visualization section. After reviewing them carefully, please try to understand the data and determine which approach would be more effective—for example, whether the focus should be on the mobile application or the website.

You can also analyze the graphs to identify which method generates the most profit and attracts the most customers. Please spend some time reviewing the graphs and use them to develop your own understanding of the best approach.

```

39 #=====
40 # TRAINING MODEL
41 #=====
42 y = customers['Yearly Amount Spent']
43 x = customers[['Avg. Session Length', 'Time on App', 'Time on Website', 'Length of Membership']]
44
45 from sklearn.model_selection import train_test_split
46 X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=101)
47
48 from sklearn.linear_model import LinearRegression
49 regressor = LinearRegression()
50 regressor.fit(X_train, y_train)
51
52
53 predictions = regressor.predict(X_test)
54
55 plt.scatter(y_test, predictions)
56 plt.xlabel('Y Test')
57 plt.ylabel('Predicted Y')
58 plt.show()
59
60 # calculate these metrics by hand!
61 from sklearn import metrics
62
63 print('MAE:', metrics.mean_absolute_error(y_test, predictions))
64 print('MSE:', metrics.mean_squared_error(y_test, predictions))
65 print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, predictions)))
66

```

Machine Learning: Step-by-Step Breakdown (Lines 42–65)

To become a "zero-to-hero" in machine learning, let's break down exactly what happens in these lines. Think of this process as teaching a student (the model) to take a final exam.

Step 1: Defining Inputs and Outputs

Python

```

y = customers['Yearly Amount Spent']
x = customers[['Avg. Session Length', 'Time on App', 'Time on Website', 'Length of Membership']]

```

- **y = customers['Yearly Amount Spent']:** This line takes the specific column titled 'Yearly Amount Spent' from our table and saves it into a new variable called **y**. We call this our "Target" or "Output"—it is the answer key we want the machine to learn to predict.
- **x = customers[['Avg. Session Length', ...]]:** This line selects the four columns that we believe contain the clues (features) to predict the yearly amount spent. We save these columns into the variable **x**. This is the "Question Paper" or the inputs.

Step 2: Splitting Data into "Study" and "Test"

Python

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x, y,
test_size=0.3, random_state=101)
```

- **from sklearn.model_selection import train_test_split:** This line imports a special tool that can randomly cut our data into two separate piles: one for studying and one for testing.
- **train_test_split(x, y, test_size=0.3, random_state=101):**
 - **test_size=0.3:** This tells the computer to keep 30% of the data hidden as a "Final Exam," while using 70% to "Study."
 - **random_state=101:** This is a "seed." Because computers cannot be truly random, this number tells the computer to start the shuffling process at a specific point. This ensures that every time you run this code, the split is exactly the same, making your results consistent.

Step 3: Training the Model (The Studying Phase)

Python

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

- **from sklearn.linear_model import LinearRegression:** This imports the "Linear Regression" algorithm—the specific mathematical tool (our "student") that will learn the relationship between the inputs and outputs.
- **regressor = LinearRegression():** This line creates an empty model. Imagine this as taking a blank notebook and naming it **regressor**. It is now ready to learn.
- **regressor.fit(X_train, y_train):** The **.fit()** function is the actual act of studying. You are feeding the model the 70% of clues (**X_train**) and the correct answers (**y_train**). The model looks at these to find the mathematical pattern (e.g., how much extra spending happens per minute on the app).

Step 4: Predicting (Taking the Exam)

Python

```
predictions = regressor.predict(X_test)
```

- **predictions = regressor.predict(X_test):** Now that the student has studied, we give them the 30% of "Final Exam" questions (**X_test**) that they have never seen before. The **.predict()** function forces the model to guess the answers using the patterns it learned. We save these guesses in the variable **predictions**.

Step 5: Visualizing Results

Python

```
plt.scatter(y_test, predictions)
plt.xlabel('Y Test')
plt.ylabel('Predicted Y')
plt.show()
```

- **plt.scatter(y_test, predictions)**: This function creates a scatter chart. It plots the Actual Answers (**y_test**) on one axis and the Model's Guesses (**predictions**) on the other.
- **plt.xlabel('Y Test')**: This adds the label "Y Test" to the bottom of the graph.
- **plt.ylabel('Predicted Y')**: This adds the label "Predicted Y" to the side of the graph.
- **plt.show()**: This command opens a window to display the final graph you just built.

Step 6: Measuring Error (Grading the Exam)

Python

```
from sklearn import metrics
print('MAE:', metrics.mean_absolute_error(y_test, predictions))
print('MSE:', metrics.mean_squared_error(y_test, predictions))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test,
predictions)))
```

- **from sklearn import metrics**: This imports the "grading tools" (the metrics).
- **metrics.mean_absolute_error(y_test, predictions)**: Calculates the Mean Absolute Error (MAE). It finds the average difference between the model's guess and the actual truth.
- **metrics.mean_squared_error(y_test, predictions)**: Calculates the Mean Squared Error (MSE). This is similar to MAE, but it squares the difference, which gives a much larger penalty to big mistakes.

np.sqrt(...): This takes the Root Mean Squared Error (RMSE). By taking the square root of the MSE, we turn the number back into the same "units" (dollars) as our original data, which makes it much easier to understand how "off" our model really is.

Interpreting the Machine Learning Results

Okay, these are the results that you got from executing the three lines. Now let's interpret this.

```
60 # calculate these metrics by hand!
61 from sklearn import metrics
62
63 print('MAE:', metrics.mean_absolute_error(y_test, predictions))
64 print('MSE:', metrics.mean_squared_error(y_test, predictions))
65 print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, predictions)))
66
```

Output for this code:

None

MAE: 7.228148653430811

MSE: 79.81305165097385

RMSE: 8.9338150669786

1. What does **None** mean?

None is **not** an error from the machine learning model. It usually appears because a function such as `plt.show()` does not return a value. Therefore, for model evaluation, we can simply **ignore None**.

2. MAE = 7.23

MAE stands for **Mean Absolute Error**. It tells us, on average, how far the model's predictions are from the actual values.

Your MAE is:

7.237.23

This means that the model's predictions are, on average, about **7.23 units away from the actual yearly spending**.

For example, if the actual amount is **\$500**, the model might predict around **\$493 or \$507**. The average difference between the predictions and the actual values is approximately **7.23**.

Therefore, a **lower MAE generally means a better model**.

3. MSE = 79.81

MSE stands for **Mean Squared Error**. It measures the prediction errors after **squaring them**.

Your MSE is:

79.8179.81

The important thing to remember is that we should **not interpret this as the model being wrong by \$79.81**. Because the errors are squared, the number is in squared units.

MSE gives more importance to **larger errors**. For example:

$$10^2 = 100$$

while:

$$2^2 = 4$$

So, larger mistakes have a much greater effect on MSE.

Again, **a lower MSE generally indicates better performance**.

4. RMSE = 8.93

RMSE stands for **Root Mean Squared Error**. It is obtained by taking the square root of the MSE:

$$RMSE = \sqrt{MSE}$$

In your case:

$$RMSE = \sqrt{79.81} \approx 8.93$$

This tells us that the model's prediction error is approximately **8.93 units**, while giving more weight to larger errors.

RMSE is especially useful because, unlike MSE, it is expressed in approximately the **same units as the value being predicted**, making it easier to understand.

Overall Conclusion

Your **MAE of 7.23** and **RMSE of 8.93** are relatively small compared with yearly spending values in the hundreds. This suggests that the model is making predictions that are **fairly close to the actual values**. Therefore, this Linear Regression model appears to be **a good model for predicting yearly customer spending with this dataset**.

Deciding the answer

```
68
69 coefficients = pd.DataFrame(regressor.coef_, x.columns)
70 coefficients.columns = ['Coefficient']
71 print(coefficients)
72
```

These three lines are the most important lines. One of these three lines are important lines in this code because these three lines command this module to give the coefficients of the predicted values. And we use these coefficients to determine our answer.

Firstly, let's just understand what is these three lines of code doing:

These lines perform the final step of extracting and displaying the mathematical weights the model learned during training. Here is the breakdown of what each line does:

- `coefficients = pd.DataFrame(regressor.coef_, x.columns)`: This line creates a new pandas DataFrame to store the coefficients. It takes `regressor.coef_` (the actual numerical weights the model calculated for each feature) and pairs them with `x.columns` (the names of your input features, like "Time on App" or "Time on Website"). This organizes the data into a readable table format.
- `coefficients.columns = ['Coefficient']`: This line simply renames the column in the new DataFrame to "Coefficient," making it clear what the numerical values represent when you look at the table.
- `print(coefficients)`: This outputs the table to your console, allowing you to see the final coefficient values so you can analyze which features have the strongest impact on the "Yearly Amount Spent".

Interpreting the Coefficients

The coefficients tell us how strongly each feature is related to the customer's **Yearly Amount Spent**. More specifically, a coefficient tells us how much the **predicted yearly spending changes when that particular feature increases by one unit, while all the other features remain constant**.

Avg. Session Length → 25.981550
Time on App → 38.590159
Time on Website → 0.190405
Length of Membership → 61.279097

1. Avg. Session Length → 25.98

When the average session length increases by **one unit**, the model predicts that the customer's yearly spending will increase by approximately **25.98 units**, assuming the other variables remain unchanged.

2. Time on App → 38.59

When the customer's time on the app increases by **one unit**, the predicted yearly spending increases by approximately **38.59 units**, assuming the other variables stay the same.

This is a relatively high positive coefficient, which means that **Time on App has a strong positive relationship with yearly spending** in this model.

3. Time on Website → 0.19

When the customer's time on the website increases by **one unit**, the predicted yearly spending increases by only approximately **0.19 units**, assuming the other variables remain constant.

Compared with the app coefficient of **38.59**, this is a very small value. Therefore, in this model, **Time on App has a much stronger relationship with yearly spending than Time on Website**.

4. Length of Membership → 61.28

This is the **largest coefficient** among the four variables. It means that increasing the length of membership by one unit is associated with an increase of approximately **61.28 units in predicted yearly spending**, assuming the other variables remain unchanged.

Important Correction

We should **not say that a higher coefficient automatically means that the company earns more revenue from that feature**.

For example:

Time on App → 38.59

Time on Website → 0.19

This tells us that, according to the model, changes in app usage are **much more strongly associated with yearly spending** than changes in website usage.

However, it does **not** mean that the app directly generates \$38.59 of revenue for every hour of use. The coefficient represents a relationship between the feature and the predicted yearly spending; it is not a direct measurement of revenue generated by that platform.

Should We Focus on the App or the Website?

This is where the answer becomes interesting: **there is no single answer based only on these coefficients**.

One person might say:

"The app has a much higher coefficient, so we should focus most of our resources on the mobile app."

This is a reasonable conclusion because the model shows a much stronger relationship between app usage and yearly spending.

However, another person might say:

"Customers already spend more through the app, so we should invest more in the website and try to improve its performance."

This can also be a reasonable business strategy.

Therefore, **the machine learning model does not directly tell us which platform the company should choose**. It tells us which variables have stronger relationships with yearly spending.

To make the final business decision, we would need additional information such as:

- Number of customers using each platform
- Operating and development costs
- Conversion rates
- Customer engagement
- Growth potential
- Expected return from investing in each platform

Final Conclusion

Based on the coefficients alone, **Time on App (38.59) has a much stronger positive relationship with yearly spending than Time on Website (0.19)**.

However, we cannot conclude from this alone that the company should completely abandon the website or focus entirely on the app.

The correct answer depends on the business situation and the additional information available.

The important lesson is:

Machine learning can tell us what the data suggests, but the final business decision still requires human judgment.